

Autonomous Drone Racing with Deep Reinforcement Learning

Yunlong Song*, Mats Steinweg*, Elia Kaufmann, and Davide Scaramuzza

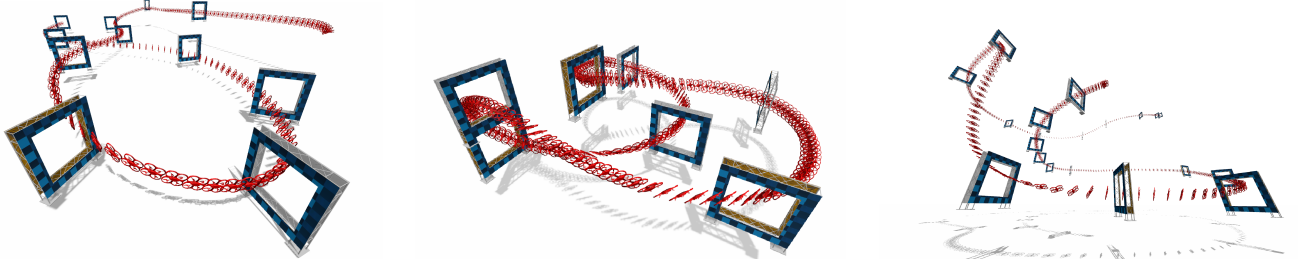


Fig. 1: A quadrotor navigates at high speed through three different race tracks: *AlphaPilot* (left), *Split-S* (middle), and *AirSim* (right). The trajectories are computed using neural network policies trained with deep reinforcement learning.

Abstract—In many robotic tasks, such as autonomous drone racing, the goal is to travel through a set of waypoints as fast as possible. A key challenge for this task is planning the time-optimal trajectory, which is typically solved by assuming perfect knowledge of the waypoints to pass in advance. The resulting solution is either highly specialized for a single-track layout, or suboptimal due to simplifying assumptions about the platform dynamics. In this work, a new approach to near-time-optimal trajectory generation for quadrotors is presented. Leveraging deep reinforcement learning and relative gate observations, our approach can compute near-time-optimal trajectories and adapt the trajectory to environment changes. Our method exhibits computational advantages over approaches based on trajectory optimization for non-trivial track configurations. The proposed approach is evaluated on a set of race tracks in simulation and the real world, achieving speeds of up to 60 km h^{-1} with a physical quadrotor.

Video: <https://youtu.be/Hebpmadjqn8>.

I. INTRODUCTION

In recent years, research on fast navigation of autonomous quadrotors has made tremendous progress, continually pushing the vehicles to more aggressive maneuvers [1]–[3]. To further advance the field, several competitions have been organized—such as the autonomous drone racing series at the recent IROS and NeurIPS conferences [4]–[7] and the AlphaPilot challenge [8], [9]—with the goal to develop autonomous systems that will eventually outperform expert human pilots.

*These authors contributed equally. The authors are with the Robotics and Perception Group, Department of Informatics, University of Zurich, and Department of Neuroinformatics, University of Zurich and ETH Zurich, Switzerland (<http://rpg.ifi.uzh.ch>). M. Steinweg is also with RWTH Aachen University. This work was supported by the National Centre of Competence in Research (NCCR) Robotics through the Swiss National Science Foundation (SNSF) and the European Union’s Horizon 2020 Research and Innovation Programme under grant agreement No. 871479 (AERIAL-CORE) and the European Research Council (ERC) under grant agreement No. 864042 (AGILEFLIGHT).

Pushing quadrotors to their physical limits raises challenging research problems, including planning a time-optimal trajectory that passes through a sequence of gates. Intuitively, such a time-optimal trajectory explores the boundary of the platform’s performance envelope. It is imperative that the planned trajectory satisfies all constraints imposed by the actuators, since the diminishing control authority will otherwise lead to catastrophic crashes. Furthermore, as quadrotors’ linear and angular dynamics are coupled, computing time-optimal trajectories requires a trade-off between maximizing linear and angular accelerations.

To generate time-optimal quadrotor trajectories, previous work has either relied on numerical optimization by discretizing the trajectory in time [1], expressed the state evolution via polynomials [10], or even approximated the quadrotor as a point mass [9]. While the optimization-based method can find trajectories that constantly push the platform to its limits, it requires very long computation times in the order of hours. In contrast, approaches that use a polynomial representation [10] or a point mass approximation [9] achieve faster computation times, but fail to account for the true actuation limits of the platform, leading to either suboptimal or dynamically infeasible solutions.

In light of its recent successes in the robotics domain [11]–[13], deep reinforcement learning (RL) has the potential to solve time-optimal trajectory planning for quadrotors. Particularly, model-free policy search learns a parametric policy by directly interacting with the environment and is well-suited for complicated problems that are difficult to model. Furthermore, policy search allows for optimizing high-capacity neural network policies, which can take different forms of state representations as inputs and allow for flexibility in the algorithm design. For example, the learned policy can map raw sensory observations of the robot directly to its motor commands.

Several research questions arise naturally, including how

具有深度强化学习的自主无人机竞赛

宋云龙*、Mats Steinweg*、Elia Kaufmann 和 Davide Scaramuzza

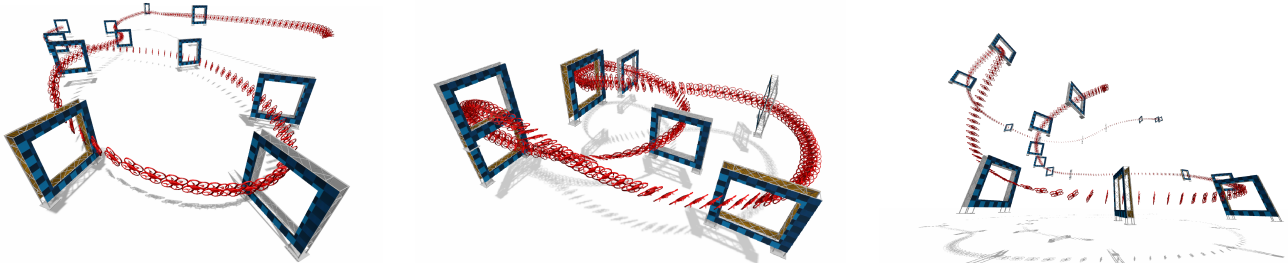


图 1: 四旋翼飞行器在三个不同的赛道上高速行驶: *AlphaPilot* (左)、*Split-S* (中)和*AirSim* (右)。轨迹是使用经过深度强化学习训练的神经网络策略来计算的。

Abstract—在许多机器人任务中,例如自主无人机竞赛,目标是尽可能快地穿过一组路径点。这项任务的一个关键挑战是规划时间最优轨迹,这通常是通过假设提前完全了解要经过的航路点来解决的。由此产生的解决方案要么是高度专业化的单轨布局,要么是由于简化了有关平台动态的假设而不是最佳的。在这项工作中,提出了一种四旋翼飞行器近时最优轨迹生成的新方法。利用深度强化学习和相对门观察,我们的方法可以计算近时间最优轨迹并使轨迹适应环境变化。与基于非平凡轨道配置的轨迹优化的方法相比,我们的方法表现出计算优势。所提出的方法在模拟和现实世界的一组赛道上进行了评估,使用物理四旋翼飞行器实现了高达 60 km h^{-1} 的速度。

视频: <https://youtu.be/Hebpmadjqn8>。

一、简介

近年来,自主四旋翼飞行器快速导航的研究取得了巨大进展,不断推动车辆进行更激进的机动[1]-[3]。为了进一步推动该领域的发展,已经组织了多项竞赛,例如最近的 IROS 和 NeurIPS 会议上的自主无人机竞赛系列 [4]-[7] 以及 AlphaPilot 挑战赛 [8]、[9],其目标是开发最终超越人类专家飞行员的自主系统。

*这些作者的贡献是平等的。作者来自苏黎世大学信息学系机器人与感知小组以及苏黎世大学和瑞士苏黎世联邦理工学院神经信息学系 (<http://rpg.ifi.uzh.ch>)。M. Steinweg 也在亚琛工业大学工作。这项工作得到了国家机器人研究中心 (NCCR) 通过瑞士国家科学基金会 (SNSF) 和欧盟地平线 2020 研究与创新计划 (第 871479 号拨款协议 (AERIAL-CORE)) 和欧洲研究理事会 (ERC) (第 864042 号拨款协议 (AGILEFLIGHT)) 的支持。

将四旋翼飞行器推向物理极限会带来具有挑战性的研究问题,包括规划穿过一系列门的时间最优轨迹。直观上,这样的时间最优轨迹探索了平台性能范围的边界。规划的轨迹必须满足执行器施加的所有约束,否则控制权的减弱将导致灾难性的事故。此外,由于四旋翼飞行器的线性和角动力学是耦合的,计算时间最优轨迹需要在最大化线性加速度和角加速度之间进行权衡。

为了生成时间最优的四旋翼飞行器轨迹,之前的工作要么依赖于时间上离散化轨迹的数值优化[1],要么通过多项式表达状态演化[10],甚至将四旋翼飞行器近似为点质量[9]。虽然基于优化的方法可以找到不断将平台推向极限的轨迹,但它需要非常长的计算时间(数小时)。相比之下,使用多项式表示[10]或点质量近似[9]的方法可以实现更快的计算时间,但无法考虑平台的真实驱动限制,从而导致解决方案不是最优的或动态不可行的。

鉴于深度强化学习 (RL) 最近在机器人领域取得的成功[11]-[13],它有潜力解决四旋翼飞行器的时间最优轨迹规划问题。特别是,无模型策略搜索通过直接与环境交互来学习参数策略,非常适合难以建模的复杂问题。此外,策略搜索允许优化高容量神经网络策略,它可以采用不同形式的状态表示作为输入,并允许算法设计的灵活性。例如,学习的策略可以将机器人的原始感官观察直接映射到其运动命令。

自然而然地出现了几个研究问题,包括如何

we can tackle the time-optimal trajectory planning problem using deep RL and how the performance compares against model-based approaches.

Contributions

Our main contribution is a reinforcement-learning-based system that can compute extremely aggressive trajectories that are close to the time-optimal solutions provided by state-of-the-art trajectory optimization [1]. To the best of our knowledge, this is the first learning-based approach for tackling time-optimal quadrotor trajectory planning that is adaptive to environmental changes. We conduct a large number of experiments, including planning on deterministic tracks, tracks with large uncertainties, and randomly generated tracks.

Our empirical results indicate that: 1) deep RL can solve the trajectory planning problem effectively, but sacrifices the performance guarantees that trajectory optimization can provide, 2) the learned neural network policy can handle large track changes and replan trajectories online, which is very challenging and computationally expensive for trajectory-optimization-based approaches, 3) the generalization capability demonstrated by our neural network policy allows for adaptation to different racing environments, suggesting that our policy could be deployed for online time-optimal trajectory generation in dynamic and unstructured environments.

The key ingredients of our approach are three-fold: 1) we use appropriate state representations, namely, the relative gate observations, as the policy inputs, 2) leveraging a highly parallelized sampling scheme, and 3) maximizing a three-dimensional path progress reward, which is a proxy of minimizing the lap time.

II. RELATED WORK

Existing works on time-optimal trajectory planning can be divided into sampling-based and optimization-based approaches. Sampling-based algorithms, such as RRT* [14], are provably optimal in the limit of infinite samples. As such algorithms rely on the construction of a graph, they are commonly combined with approaches that provide a closed-form solution for state-to-state trajectories. Many challenges exist in this line of work; for example, the closed-form solution between two full quadrotor states is difficult to derive analytically when considering the actuator constraints. For this reason, prior works [9], [15]–[17] have favoured point-mass approximations or polynomial representations for the state evolution, which possibly lead to infeasible solutions for the former and inherently sub-optimal solutions for the latter as shown in [1].

The time-optimal planning for quadrotor waypoints [1], solve the problem by using discrete-time state space representations for the trajectory, and enforcing the system dynamics and thrust limits as constraints. A major advantage of optimization-based methods, such as [1], [10], [18], is that they can incorporate nonlinear dynamics and handle a variety of state and input constraints. For passing through multiple waypoints, the allocation of the traversal times

is a priori unknown, rendering the problem formulation significantly more complicated. The requirement of solving a complex constrained optimization problem makes most trajectory optimization only suitable for applications where the computation of time-optimal trajectories can be done offline.

Deep RL has been applied to autonomous navigation tasks, such as high-speed racing [19], [20] and motion planning of autonomous vehicles [21], [22]. The main focus of these methods was to solve navigation problems for wheeled robots, whose motion is constrained to a two-dimensional plane. However, finding a time-optimal trajectory for quadrotors is more challenging since the search space is significantly larger due to the high-dimensional state and input space. Previous works applying RL to quadrotor control have shown successful waypoints following [12] and autonomous landing [23], but neither of these works managed to push the platform to its physical limits. Our work tackles the problem of racing a quadrotor through a set of waypoints in minimum time using deep RL. Our policy does not impose strong priors on the track layout, manifesting its applicability to dynamic environments.

III. METHODOLOGY

A. Quadrotor Dynamics

We model the quadrotor as a 6 degree-of-freedom rigid body of mass m and diagonal moment of inertia matrix $\mathbf{J}$. The dynamics of the system can be written as:

$$\begin{aligned}\dot{\mathbf{p}}_{WB} &= \mathbf{v}_{WB} & \dot{\mathbf{q}}_{WB} &= \frac{1}{2}\mathbf{\Lambda}(\boldsymbol{\omega}_B) \cdot \mathbf{q}_{WB} \\ \dot{\mathbf{v}}_{WB} &= \mathbf{q}_{WB} \odot \mathbf{c} + \mathbf{g} & \dot{\boldsymbol{\omega}}_B &= \mathbf{J}^{-1}(\boldsymbol{\eta} - \boldsymbol{\omega}_B \times \mathbf{J}\boldsymbol{\omega}_B)\end{aligned}$$

where $\mathbf{p}_{WB} = [p_x, p_y, p_z]^T$ and $\mathbf{v}_{WB} = [v_x, v_y, v_z]^T$ are the position and the velocity vectors of the quadrotor in the world frame W . We use a unit quaternion $\mathbf{q}_{WB} = [q_w, q_x, q_y, q_z]^T$ to represent the orientation of the quadrotor and $\boldsymbol{\omega}_B = [\omega_x, \omega_y, \omega_z]^T$ to denote the body rates in the body frame B . Here, $\mathbf{g} = [0, 0, -g_z]^T$ with $g_z = 9.81 \text{ m s}^{-2}$ is the gravity vector, $\mathbf{J}$ is the inertia matrix, $\boldsymbol{\eta}$ is the three dimensional torque, and $\mathbf{\Lambda}(\boldsymbol{\omega}_B)$ is a skew-symmetric matrix. Finally, $\mathbf{c} = [0, 0, c]^T$ is the mass-normalized thrust vector. The conversion of single rotor thrusts $[f_1, f_2, f_3, f_4]$ to mass-normalized thrust c and body torques $\boldsymbol{\eta}$ is formulated as

$$c = \frac{1}{m} \sum_{i=1}^4 f_i, \quad \boldsymbol{\eta} = \begin{bmatrix} l/\sqrt{2}(f_1 - f_2 - f_3 + f_4) \\ l/\sqrt{2}(-f_1 - f_2 + f_3 + f_4) \\ \kappa(f_1 - f_2 + f_3 - f_4) \end{bmatrix},$$

where m is the quadrotor's mass, κ is the rotor's torque constant, and l is the arm length. The full state of the quadrotor is defined as $\mathbf{x} = [\mathbf{p}_{WB}, \mathbf{q}_{WB}, \mathbf{v}_{WB}, \boldsymbol{\omega}_B]$ and the control inputs are $\mathbf{u} = [f_1, f_2, f_3, f_4]$. We use a 4th order Runge-Kutta method for the numerical integration of the dynamic equation, denoted as $f_{\text{RK4}}(\mathbf{x}, \mathbf{u}, d_t)$, with d_t as the integration time step.

我们可以使用深度强化学习来解决时间最优轨迹规划问题，并了解其性能与基于模型的方法的比较。

Contributions

我们的主要贡献是基于强化学习的系统，它可以计算极其激进的轨迹，接近最先进的轨迹优化 [1] 提供的时间最优解决方案。据我们所知，这是第一个基于学习的方法，用于解决适应环境变化的时间最优四旋翼飞行器轨迹规划。我们进行了大量的实验，包括规划确定性轨迹、具有较大不确定性的轨迹以及随机生成的轨迹。

我们的实证结果表明：1) 深度强化学习可以有效地解决轨迹规划问题，但牺牲了轨迹优化可以提供的性能保证，2) 学习的神经网络策略可以处理大的赛道变化并在线重新规划轨迹，这对于基于轨迹优化的方法来说非常具有挑战性且计算成本昂贵，3) 我们的神经网络策略所展示的泛化能力允许适应不同的赛车环境，这表明我们的策略可以部署用于在线时间最优动态和非结构化环境中的轨迹生成。

我们的方法的关键要素有三个：1) 我们使用适当的状态表示，即相对门观察，作为策略输入，2) 利用高度并行化的采样方案，3) 最大化三维路径进度奖励，这是最小化单圈时间的代理。

二.相关工作

现有的时间最优轨迹规划工作可以分为基于采样的方法和基于优化的方法。基于采样的算法，例如 RRT* [14]，在无限样本的限制下被证明是最优的。由于此类算法依赖于图的构造，因此它们通常与为状态到状态轨迹提供封闭式解决方案的方法相结合。这项工作存在许多挑战；例如，在考虑执行器约束时，很难通过分析推导两个完整四旋翼飞行器状态之间的封闭解。因此，先前的工作[9]、[15]-[17]倾向于对状态演化采用点质量近似或多项式表示，这可能导致前者的解不可行，而后者本质上是次优解，如[1]所示。

四旋翼飞行器航路点的时间最优规划[1]，通过使用轨迹的离散时间状态空间表示，并强制系统动力学和推力限制作为约束来解决问题。基于优化的方法（例如[1]、[10]、[18]）的主要优点是它们可以结合非线性动力学并处理各种状态和输入约束。对于经过多个路点，遍历时间的分配

是先验未知的，使得问题的表述变得更加复杂。解决复杂约束优化问题的要求使得大多数轨迹优化仅适用于可以离线完成时间最优轨迹计算的应用。

深度强化学习已应用于自主导航任务，例如高速赛车[19]、[20]和自动驾驶车辆的运动规划[21]、[22]。这些方法的主要重点是解决轮式机器人的导航问题，其运动被限制在二维平面内。然而，为四旋翼飞行器找到时间最优轨迹更具挑战性，因为高维状态和输入空间导致搜索空间显著增大。之前将强化学习应用于四旋翼飞行器控制的工作已经展示了成功的航路点跟踪[12]和自主着陆[23]，但这些工作都没有成功地将平台推向其物理极限。我们的工作解决了使用深度强化学习在最短时间内让四旋翼飞行器通过一组航路点的问题。我们的政策并没有对赛道布局强加强加的优先性，这表明了它对动态环境的适用性。

三. 方法论

A. Quadrotor Dynamics

我们将四旋翼飞行器建模为具有质量 m 和对角惯性矩矩阵 $\mathbf{J}$ 的 6 自由度刚体。系统的动力学可以写为：

$$\begin{aligned}\dot{\mathbf{p}}_{WB} &= \mathbf{v}_{WB} & \dot{\mathbf{q}}_{WB} &= \frac{1}{2} \mathbf{\Lambda}(\boldsymbol{\omega}_B) \cdot \mathbf{q}_{WB} \\ \dot{\mathbf{v}}_{WB} &= \mathbf{q}_{WB} \odot \mathbf{c} + \mathbf{g} & \dot{\boldsymbol{\omega}}_B &= \mathbf{J}^{-1}(\boldsymbol{\eta} - \boldsymbol{\omega}_B \times \mathbf{J} \boldsymbol{\omega}_B)\end{aligned}$$

其中 $\mathbf{p}_{WB} = [p_x, p_y, p_z]^T$ 和 $\mathbf{v}_{WB} = [v_x, v_y, v_z]^T$ 是四旋翼飞行器在世界坐标系 W 中的位置和速度向量。我们使用单位四元数 $\mathbf{q}_{WB} = [q_w, q_x, q_y, q_z]^T$ 表示四旋翼飞行器的方向，使用 $\boldsymbol{\omega}_B = [\omega_x, \omega_y, \omega_z]^T$ 表示机身坐标系 B 中的机身速率。其中， $\mathbf{g} = [0, 0, -g_z]^T$ 和 $g_z = 9.81 \text{ m s}^{-2}$ 是重力矢量， $\mathbf{J}$ 是惯性矩阵， $\boldsymbol{\eta}$ 是三维扭矩， $\mathbf{\Lambda}(\boldsymbol{\omega}_B)$ 是斜对称矩阵。最后， $\mathbf{c} = [0, 0, c]^T$ 是质量归一化推力矢量。单转子推力 $[f_1, f_2, f_3, f_4]$ 到质量归一化推力 c 和机身扭矩 $\boldsymbol{\eta}$ 的转换公式如下：

$$c = \frac{1}{m} \sum_{i=1}^4 f_i, \quad \boldsymbol{\eta} = \begin{bmatrix} l/\sqrt{2}(f_1 - f_2 - f_3 + f_4) \\ l/\sqrt{2}(-f_1 - f_2 + f_3 + f_4) \\ \kappa(f_1 - f_2 + f_3 - f_4) \end{bmatrix},$$

其中 m 是四旋翼飞行器的质量， κ 是转子的扭矩常数， l 是臂长。四旋翼飞行器的完整状态定义为 $\mathbf{x} = [\mathbf{p}_{WB}, \mathbf{q}_{WB}, \mathbf{v}_{WB}, \boldsymbol{\omega}_B]$ ，控制输入为 $\mathbf{u} = [f_1, f_2, f_3, f_4]$ 。我们使用四阶龙格-库塔方法对动力学方程进行数值积分，记为 $f_{RK4}(\mathbf{x}, \mathbf{u}, d_t)$ ，以 d_t 作为积分时间步长。

B. Task Formulation

We formulate the time-optimal trajectory planning problem in the reinforcement learning framework. To this end, we model the task using an infinite-horizon Markov Decision Process (MDP) defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \rho_0, \gamma)$ [24]. The RL agent starts in a state $s_t \in \mathcal{S}$ drawn from an initial state distribution $\rho_0(s)$. At every time step t , an action $a_t \in \mathcal{A}$ sampled from a stochastic policy $\pi(a_t|s_t)$ is executed and the agent transitions to the next state s_{t+1} with the state transition probability $\mathcal{P}_{s_t s_{t+1}}^{a_t} = \Pr(s_{t+1}|s_t, a_t)$, receiving an immediate reward $r(t) \in \mathbb{R}$. Explicit representations of state and action are discussed in Section III-B.2.

The goal of deep RL is to optimize the parameters θ of a neural network policy π_θ such that the trained policy maximizes the expected discounted reward over an infinite horizon. The discrete time-formulation of the objective is

$$\pi_\theta^* = \operatorname{argmax}_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(t) \right] \quad (1)$$

where $\gamma \in [0, 1)$ is the discount factor that trades off long-term against short-term rewards. The optimal trajectory τ^* is obtained by rolling out the trained policy.

1) *Reward Function*: Intuitively, the total time spent on the track directly captures the main objective of the task. However, this signal can only be computed upon the successful completion of a full lap, introducing sparsity that considerably increases the difficulty of the credit assignment to individual actions. A popular approach to circumvent this problem is to use a proxy reward that closely approximates the true performance objective while providing feedback to the agent at every time step. In car racing, reward shaping techniques based on the projected progress along the center-line of the race track have shown to work well as an approximation of the true racing reward [19], [25].

We extend the concept of a projection-based path progress reward to the task of drone racing by using straight line segments connecting adjacent gate centers as a representation of the track center-line. Thus, no additional effort is required for computing the reference path. A race track is completely defined by a set of gates placed in three-dimensional space.

At any point in time, the quadrotor can be associated with a specific line segment based on the next gate to be passed. To compute the path progress, we project the position of the quadrotor onto the current line segment. Given the current position $\mathbf{p}_c(t)$ and previous position $\mathbf{p}_c(t-1)$ of the quadrotor, we define the progress reward $r_p(t)$ as follows:

$$r_p(t) = s(\mathbf{p}_c(t)) - s(\mathbf{p}_c(t-1)). \quad (2)$$

Here, $s(\mathbf{p}) = (\mathbf{p} - \mathbf{g}_1) \cdot (\mathbf{g}_2 - \mathbf{g}_1) / \|\mathbf{g}_2 - \mathbf{g}_1\|$ defines the progress along the path segment that connects the previous gate's center $\mathbf{g}_1$ with the next gate's center $\mathbf{g}_2$.

In addition, we define a safety reward to incentivize the vehicle to fly through the gate center by penalizing small safety margins. It should be noted that the safety reward is an optional reward component designed to reduce the risk of

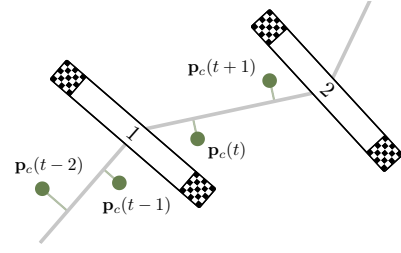


Fig. 2: Illustration of the progress reward.

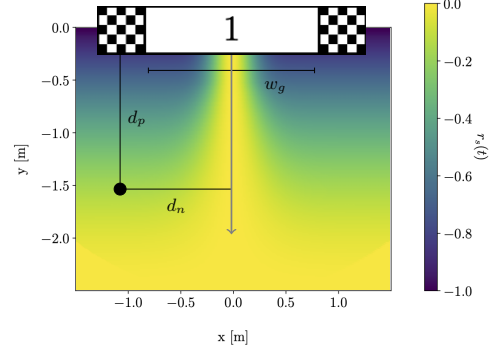


Fig. 3: Illustration of the safety reward.

crashing in training settings that feature large track changes. The safety reward is defined as follows:

$$r_s(t) = -f^2 \cdot \left(1 - \exp \left(-\frac{0.5 \cdot d_n^2}{v} \right) \right) \quad (3)$$

where $f = \max [1 - (d_p/d_{\max}), 0.0]$ and $v = \max [(1 - f) \cdot (w_g/6), 0.05]$. Here, d_p and d_n denote the distance of the quadrotor to the gate normal and the distance to the gate plane, respectively. The distance to the gate normal is normalized by the side length of the rectangular gate w_g , while $d_{\max}$ specifies a threshold on the distance to the gate center in order to activate the safety reward. We illustrate the safety reward in Figure 3.

The final reward at each time step t is defined as

$$r(t) = r_p(t) + a \cdot r_s(t) - b \cdot \|\omega_t\|^2 + \begin{cases} r_T & \text{if gate collision} \\ 0 & \text{else} \end{cases} \quad (4)$$

where r_T is a terminal reward for penalizing gate collisions

$$r_T = -\min \left[\left(\frac{d_g}{w_g} \right)^2, 20.0 \right] \quad (5)$$

with d_g and w_g denoting the euclidean distance from the position of the crash to the gate center and the side length of the rectangular gate, respectively. Furthermore, a quadratic penalty on the body rates ω weighted by $b \in \mathbb{R}$ is added at an initial training stage. Here, $a \in \mathbb{R}$ is a hyperparameter that trades off between progress maximization and risk minimization.

2) *Observation and Action Spaces*: The observation space consists of two main components, one related to the quadrotor's state s_t^{quad} and one capturing information about the race track s_t^{track} . We define the observation vector for the quadrotor state as $s_t^{\text{quad}} = [\mathbf{v}_{WB,t}, \dot{\mathbf{v}}_{WB,t}, \mathbf{R}_{WB,t}, \omega_{B,t}] \in \mathbb{R}^{18}$,

B. Task Formulation

我们在强化学习框架中制定了时间最优轨迹规划问题。为此，我们使用由元组 $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \rho_0, \gamma)$ 定义的无限范围马尔可夫决策过程 (MDP) 对任务进行建模 [24]。RL 代理从初始状态分布 $\rho_0(s)$ 得出的状态 $s_t \in \mathcal{S}$ 开始。在每个时间步 t ，执行从随机策略 $\pi(a_t|s_t)$ 采样的动作 $a_t \in \mathcal{A}$ ，并且智能体以状态转移概率 $\mathcal{P}_{s_t, s_{t+1}}^{a_t} = \Pr(s_{t+1}|s_t, a_t)$ 转换到下一个状态 s_{t+1} ，并接收即时奖励 $r(t) \in \mathbb{R}$ 。状态和动作的明确表示将在第 III-B.2 节中讨论。

深度强化学习的目标是优化神经网络策略 π_θ 的参数 θ ，以便训练后的策略在无限范围内最大化预期折扣奖励。目标的离散时间表述为

$$\pi_\theta^* = \operatorname{argmax}_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(t) \right] \quad (1)$$

其中 $\gamma \in [0, 1)$ 是权衡长期回报和短期回报的折扣因子。最优轨迹 τ^* 是通过推出经过训练的策略获得的。

1) *Reward Function*: 直观上，在赛道上花费的总时间直接反映了任务的主要目标。然而，这个信号只能在成功完成一圈后才能计算，从而引入稀疏性，从而大大增加了对个人动作进行信用分配的难度。规避此问题的一种流行方法是使用接近真实性能目标的代理奖励，同时，在每个时间步骤向代理提供反馈。在赛车中，基于沿赛道中心线的预计进度的奖励塑造技术已被证明可以很好地近似真实的赛车奖励 [19], [25]。

我们通过使用连接相邻门中心的直线段作为赛道中心线的表示，将基于投影的路径进度奖励的概念扩展到无人机竞赛任务。因此，不需要额外的努力来计算参考路径。赛道完全由放置在三维空间中的一组门定义。

在任何时间点，四旋翼飞行器都可以根据下一个要通过的门与特定的线段相关联。为了计算路径进度，我们将四旋翼飞行器的位置投影到当前线段上。给定四旋翼飞行器的当前位置 $\mathbf{p}_c(t)$ 和先前位置 $\mathbf{p}_c(t-1)$ ，我们定义进度奖励 $r_p(t)$ 如下：

$$r_p(t) = s(\mathbf{p}_c(t)) - s(\mathbf{p}_c(t-1)). \quad (2)$$

这里， $s(\mathbf{p}) = (\mathbf{p} - \mathbf{g}_1) \cdot (\mathbf{g}_2 - \mathbf{g}_1) / \|\mathbf{g}_2 - \mathbf{g}_1\|$ 定义了沿着连接前一个门的中心 $\mathbf{g}_1$ 和下一个门的中心 $\mathbf{g}_2$ 的路径段的进度。

此外，我们还定义了安全奖励，通过惩罚较小的安全裕度来激励车辆飞过大门中心。需要注意的是，安全奖励是一个可选的奖励成分，旨在降低安全风险

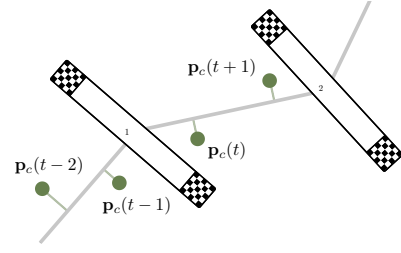


图 2：进度奖励说明。

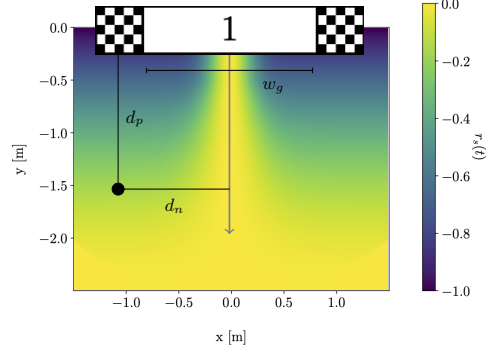


图 3：安全奖励说明。

在赛道变化较大的训练环境中崩溃。安全奖励定义如下：

$$r_s(t) = -f^2 \cdot \left(1 - \exp \left(-\frac{0.5 \cdot d_n^2}{v} \right) \right) \quad (3)$$

其中 $f = \max [1 - (d_p/d_{\max}), 0.0]$ 和 $v = \max [(1 - f) \cdot (w_g/6), 0.05]$ 。这里， d_p 和 d_n 分别表示四旋翼到门法线的距离和到门平面的距离。到门法线的距离由矩形门 w_g 的边长标准化，而 $d_{\max}$ 指定到门中心距离的阈值，以激活安全奖励。我们在图 3 中说明了安全奖励。

每个时间步 t 的最终奖励定义为

$$r(t) = r_p(t) + a \cdot r_s(t) - b \cdot \|\omega_t\|^2 + \begin{cases} r_T & \text{如果门碰撞} \\ 0 & \text{别的} \end{cases} \quad (4)$$

其中 r_T 是惩罚门碰撞的最终奖励

$$r_T = -\min \left[\left(\frac{d_g}{w_g} \right)^2, 20.0 \right] \quad (5)$$

其中 d_g 和 w_g 分别表示碰撞位置到门中心的欧氏距离和矩形门的边长。此外，在初始训练阶段添加对由 $b \in \mathbb{R}$ 加权的身体速率 ω 的二次惩罚。这里， $a \in \mathbb{R}$ 是一个超参数，在进度最大化和风险最小化之间进行权衡。

2) *Observation and Action Spaces*: 观察空间由两个主要部分组成，一个与四旋翼飞行器的状态 $\mathbf{s}_t^{\text{quad}}$ 相关，另一个捕获有关赛道 $\mathbf{s}_t^{\text{track}}$ 的信息。我们将四旋翼飞行器状态的观测向量定义为 $\mathbf{s}_t^{\text{quad}} = [\mathbf{v}_{WB,t}, \dot{\mathbf{v}}_{WB,t}, \mathbf{R}_{WB,t}, \omega_{B,t}] \in \mathbb{R}^{18}$,

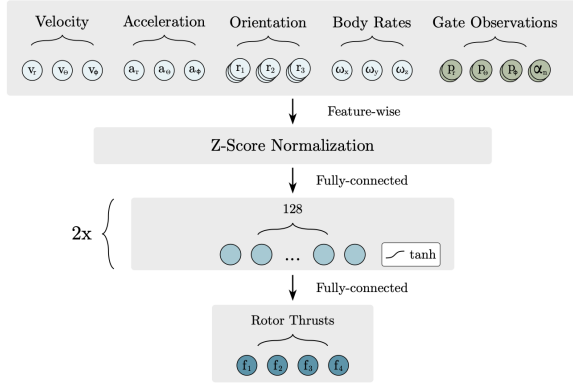


Fig. 4: Illustration of the observation vector and network architecture used in this work.

which corresponds to the quadrotor’s linear velocity, linear acceleration, rotation matrix, and angular velocity, respectively. To avoid ambiguities and discontinuities in the representation of the orientation, we use rotation matrices to describe the quadrotor attitude as was done in previous work on quadrotor control using RL [12].

We define the track observation vector as $\mathbf{s}_t^{\text{track}} = [\mathbf{o}_1, \alpha_1, \dots, \mathbf{o}_i, \alpha_i, \dots]$, $i \in [1, \dots, N]$, where $\mathbf{o}_i \in \mathbb{R}^3$ denotes the observation for gate i and $N \in \mathbb{Z}^+$ is the total number of future gates. We use spherical coordinates $\mathbf{o}_i = (p_r, p_\theta, p_\phi)_i$ to represent the observation of the next gate, as this representation provides a clear separation between the distance to the gate and the flight direction. For the next gate to be passed, we use a body-centered reference frame, while all other gate observations are recursively expressed in the frame of the previous gate. Here, $\alpha_i \in \mathbb{R}$ specifies the angle between the gate normal and the vector pointing from the quadrotor to the center of gate i . The generic formulation of the track observation allows us to incorporate a variable number of future gates into the observation. We apply input normalization using z-scoring by computing observation statistics across deterministic policy rollouts on 1000 randomly generated three-dimensional race tracks.

The agent is trained to directly map the observation to individual thrust commands, defining the action as $\mathbf{a}_t = [f_1, f_2, f_3, f_4]$. Using individual rotor commands allows for maximum versatility and aggressive flight maneuvers. We use the Tanh activation function at the last layer of the policy network to keep the control commands within a fixed range.

C. Policy Training

We train our agent using the Proximal Policy Optimization (PPO) algorithm [26], a first-order policy gradient method that is particularly popular for its good benchmark performances and simplicity in implementation. Our task presents challenges for PPO due to the high dimensionality of the search space and the complexity of the required maneuvers. We highlight three key ingredients that allow us to achieve stable and fast training performance on complex three-dimensional race tracks with arbitrary track lengths and numbers of gates:

1) *Parallel Sampling Scheme*: Training our agent in simulation allows for performing policy rollouts on up to 100

environments in parallel, resulting in a significant speed-up of the data collection process. Furthermore, the parallelization can be leveraged to increase the diversity of the collected environment interactions. When training on a long race track, initialization strategies can be designed such that rollout trajectories cover the whole race track instead of being limited to certain parts of the track. Similarly, in a setting where strong track randomization is applied, rollouts can be performed on a variety of tracks using different environments, efficiently avoiding overfitting to a certain track configuration.

2) *Distributed Initialization Strategy*: At the early stages of the learning process, a majority of the rollouts terminate on a gate crash. Consequently, if each quadrotor is initialized at the starting position, the data collection is restricted to a small part of the state space, requiring a large number of update steps until the policy is capable of exploring the full track. To counteract this situation, we employ a distributed initialization strategy that ensures a uniform exploration of relevant areas of the state space. We randomly initialize the quadrotors in a hover state around the centers of all path segments, immediately exposing them to all gate observations. Once the policy has learned to pass gates reliably, we sample initial states from previous trajectories. Thus, we retain the benefit of initializing across the whole track while avoiding the negative impact of starting from a hover position in areas of the track that are associated with high speeds.

3) *Random Track Curriculum*: Training a racing policy for a single track already constitutes a challenging problem. However, given the static nature of the track, an appropriate initialization strategy allows for solving the problem without additional guidance of the training process. When going beyond the setting in which a deterministic race track is used for training, the complexity of the task increases drastically. Essentially, training a policy by randomly sampling race tracks, confronts the agent with a new task in each episode.

For this setting, we first design a track generator by concatenating a list of randomly generated gate primitives. We mathematically define the track generator as

$$\mathbf{T} = [\mathbf{G}_1, \dots, \mathbf{G}_{j+1}], \quad j \in [1, +\infty) \quad (6)$$

where $\mathbf{G}_{j+1} = f(\mathbf{G}_j, \Delta\mathbf{p}, \Delta\mathbf{R})$ is the gate primitive, which is parameterized via the relative position $\Delta\mathbf{p}$ and orientation $\Delta\mathbf{R}$ between two consecutive gates. Thus, by adjusting the range of relative poses and the total number of gates, we can generate race tracks of arbitrary complexity and length.

To design a training process in which the focus remains on minimizing the lap time despite randomly generated tracks, we propose to automatically adapt the complexity of the sampled tracks based on the agent’s performance. Specifically, we start the training by sampling tracks with only small deviations from a straight line. The policy quickly learns to pass gates reliably, reducing the percentage of rollouts that results in a gate crash. We use the crash ratio computed across all parallel environments to adapt the complexity of the sampled tracks by allowing for more diverse relative gate poses. Thus, the data collection is constrained to a

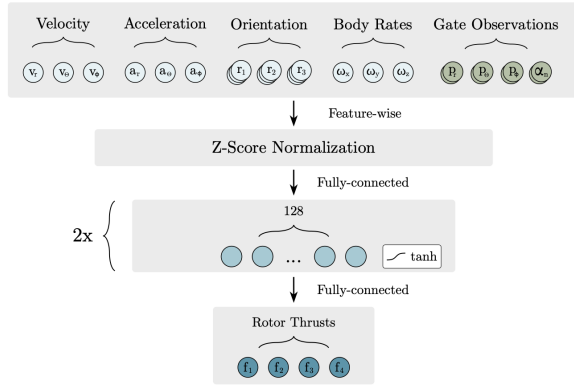


图 4: 本工作中使用的观察向量和网络架构的图示。

分别对应于四旋翼飞行器的线速度、线加速度、旋转矩阵和角速度。为了避免方向表示中的模糊性和不连续性，我们使用旋转矩阵来描述四旋翼飞行器的姿态，就像之前使用强化学习进行四旋翼飞行器控制的工作中所做的那样[12]。

我们将轨迹观测向量定义为 $\mathbf{s}_t^{\text{track}} = [\mathbf{o}_1, \alpha_1, \dots, \mathbf{o}_i, \alpha_i, \dots]$, $i \in [1, \dots, N]$, 其中 $\mathbf{o}_i \in \mathbb{R}^3$ 表示门 i 的观测, $N \in \mathbb{Z}^+$ 是未来门的总数。我们使用球坐标 $\mathbf{o}_i = (p_r, p_\theta, p_\phi)_i$ 来表示对下一个门的观察, 因为这种表示提供了到门的距离和飞行方向之间的清晰分离。对于要通过的下一个门, 我们使用以身体为中心的参考系, 而所有其他门观察都在前一个门的框架中递归表达。这里, $\alpha_i \in \mathbb{R}$ 指定门法线与从四旋翼飞行器指向门中心 i 的矢量之间的角度。轨道观察的通用公式允许我们将可变数量的未来门合并到观察中。我们通过计算 1000 个随机生成的三维赛道上的确定性策略推出的观察统计数据, 使用 z 评分应用输入标准化。

代理经过训练, 可以将观察结果直接映射到各个推力命令, 将动作定义为 $\mathbf{a}_t = [f_1, f_2, f_3, f_4]$ 。使用单独的旋翼命令可以实现最大的多功能性和激进的飞行机动。我们在策略网络的最后一层使用 Tanh 激活函数来控制命令保持在固定范围内。

C. Policy Training

我们使用近端策略优化 (PPO) 算法[26]来训练我们的代理, 这是一种一阶策略梯度方法, 因其良好的基准性能和实施简单性而特别受欢迎。由于搜索空间的高维性和所需操作的复杂性, 我们的任务对 PPO 提出了挑战。我们强调三个关键因素, 使我们能够在任意赛道长度和门数量的复杂三维赛道上实现稳定和快速的训练表现:

1) *Parallel Sampling Scheme*: 通过模拟训练我们的代理, 可以在多达 100 个对象上执行策略部署

并行环境, 从而显著加快数据收集过程。此外, 可以利用并行化来增加所收集的环境交互的多样性。在长赛道上训练时, 可以设计初始化策略, 使推出轨迹覆盖整个赛道, 而不是仅限于赛道的某些部分。类似地, 在应用强轨道随机化的设置中, 可以使用不同环境在各种轨道上执行转出, 从而有效地避免对特定轨道配置的过度拟合。

2) *Distributed Initialization Strategy*: 在学习过程的早期阶段, 大多数部署都会因门崩溃而终止。因此, 如果每个四旋翼飞行器在起始位置初始化, 则数据收集仅限于状态空间的一小部分, 需要大量更新步骤, 直到策略能够探索完整轨迹。为了应对这种情况, 我们采用分布式初始化策略, 确保对状态空间的相关区域进行统一探索。我们将四旋翼飞行器随机初始化为围绕所有路径段中心的悬停状态, 立即将它们暴露给所有门观测。一旦策略学会了可靠地通过门, 我们就会从之前的轨迹中采样初始状态。因此, 我们保留了在整个轨道上进行初始化的好处, 同时避免了在与高速相关的轨道区域中从悬停位置开始的负面影响。

3) *Random Track Curriculum*: 训练单一赛道的赛车策略已经构成了一个具有挑战性的问题。然而, 考虑到赛道的静态性质, 适当的初始化策略可以解决问题, 而无需额外的训练过程指导。当超出使用确定性赛道进行训练的设置时, 任务的复杂性会急剧增加。本质上, 通过随机采样赛道来训练策略, 让智能体在每一集中面临一个新任务。

对于此设置, 我们首先通过连接随机生成的门基元列表来设计轨道生成器。我们在数学上将轨迹生成器定义为

$$\mathbf{T} = [\mathbf{G}_1, \dots, \mathbf{G}_{j+1}], \quad j \in [1, +\infty) \quad (6)$$

其中 $\mathbf{G}_{j+1} = f(\mathbf{G}_j, \Delta \mathbf{p}, \Delta \mathbf{R})$ 是门原语, 它通过两个连续门之间的相对位置 $\Delta \mathbf{p}$ 和方向 $\Delta \mathbf{R}$ 进行参数化。因此, 通过调整相对位姿的范围和门的总数, 我们可以生成任意复杂度和长度的赛道。

为了设计一个训练过程, 尽管随机生成赛道, 但重点仍然是最小化单圈时间, 我们建议根据代理的表现自动调整采样赛道的复杂性。具体来说, 我们通过对与直线仅有很小偏差的轨迹进行采样来开始训练。该策略很快就能学会如何可靠地通过闸门, 从而减少导致闸门崩溃的推出百分比。我们使用在所有并行环境中计算的碰撞比, 通过允许更多样化的相对门姿势来适应采样轨道的复杂性。因此, 数据收集仅限于

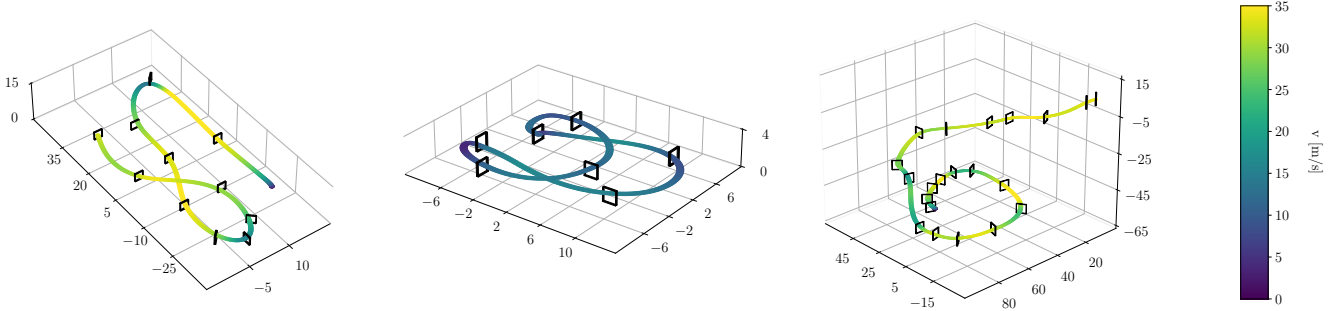


Fig. 5: Race tracks used for the baseline comparison along with trajectories generated by our approach. From left to right: *AlphaPilot* track, *Split-S* track, *AirSim* track. The colors correspond to the quadrotor’s velocity.

certain class of tracks until a specified performance threshold is reached. The adaptation of the track complexity can be interpreted as an automatic curriculum, resulting in a learning process that is tailored towards the capabilities of the agent.

IV. EXPERIMENTS

We design our experiments to answer the following research questions: (i) How does the lap time achieved by our learning-based approach compare to optimization algorithms for a deterministic track layout? (ii) How well can our approach handle changes in the track layout? (iii) Is it possible to train a policy that can successfully race on a completely unknown track? (iv) Can the trajectory generated by our policy be executed with a physical quadrotor?

A. Experimental Setup

We use the Flightmare simulator [27] and implement a vectorized OpenAI gym-style racing environment, which allows for simulating hundreds of quadrotors and race tracks in parallel. Such a parallelized implementation enables us to collect up to 25000 environment interactions per second during training. Our PPO implementation is based on [28].

To benchmark the performance of our method, we use three different race tracks, including a track from the 2019 AlphaPilot challenge [8], a track from the 2019 NeurIPS AirSim Game of Drones challenge [7], and a track designed for real-world experiments (*Split-S*). These tracks (Fig. 5) pose a diverse set of challenges for an autonomous racing drone: the *AlphaPilot* track features long straight segments and hairpin curves, the *AirSim* track requires performing rapid and very large elevation changes, and the *Split-S* track introduces two vertically stacked gates.

To test the robustness of our approach, we use random displacements of the position and the yaw angle of all gates on the *AlphaPilot* track. Furthermore, we train and evaluate our approach on completely random tracks using the track generator introduced in Section III-C.3, allowing us to study the scalability and generalizability of our approach. Finally, we validate our design choices with ablation studies and execute the generated trajectory on a physical platform. All experiments are conducted using a quadrotor configuration with diagonal inertia matrix $\mathbf{J} = [0.003, 0.003, 0.005] \text{ kg m}^2$, torque constant $\kappa = 0.01$ and arm length 0.17 m. For the

AlphaPilot and *AirSim* track, we use a thrust-to-weight ratio of 6.4 while for the *Split-S* track a thrust-to-weight ratio of 3.3 is used (for the real-world deployment). Unless noted otherwise, we perform all experiments using two future gates in the observation. We present an ablation study on the number of gates in Section IV-E.2.

B. Baseline Comparison on Deterministic Tracks

We first benchmark the performance of our policy on three deterministic tracks (Section IV-A) against two trajectory planning algorithms: polynomial minimum-snap trajectory generation [29] and optimization-based time-optimal trajectory generation based on complementary progress constraints (CPC) [1]. Polynomial trajectory generation, with the specific instantiation of minimum-snap trajectory generation, is widely used for quadrotor flight due to its implicit smoothness property. Although in general a very useful feature, the smoothness property prohibits the exploitation of the full actuator potential at every point in time, rendering the approach sub-optimal for racing. In contrast, the optimization-based method (CPC) provides an asymptotic lower bound on the possibly achievable time for a given race track and vehicle model. We do not compare to sampling-based methods due to their simplifying assumptions on the quadrotor dynamics. To the best of our knowledge, there exist no sampling-based methods that take into account the full state space of the quadrotor.

The results are summarized in Table I. Our approach achieves a performance within 5.2% of the theoretical limit illustrated by CPC. The baseline computed using polynomial trajectory generation is significantly slower. Fig. 5 illustrates the three evaluated race tracks and the corresponding trajectories generated by our approach.

TABLE I: Lap time in seconds.

Methods	AlphaPilot (s)	Split-S (s)	AirSim (s)
Polynomial [29]	12.23	15.13	23.82
Optimization [1]	8.06	6.18	11.40
Ours	8.14	6.50	11.82

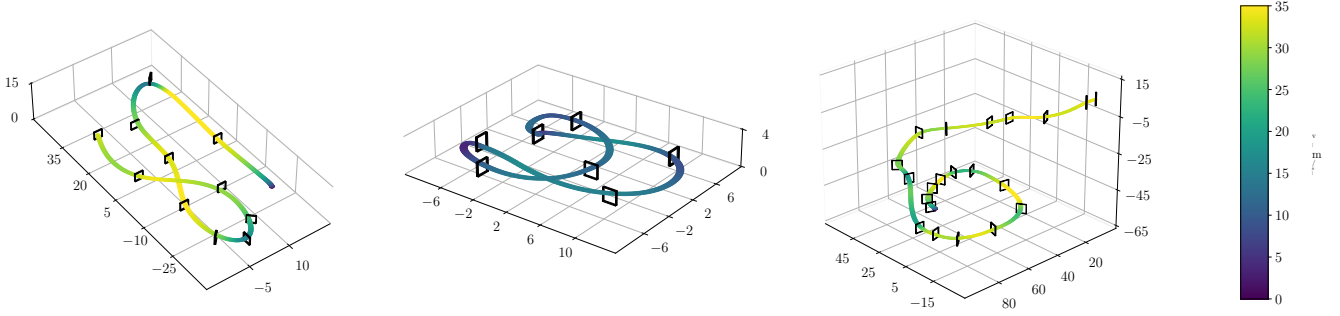


图 5: 用于基线比较的赛道以及我们的方法生成的轨迹。从左到右: *AlphaPilot* 轨道、*Split-S* 轨道、*AirSim* 轨道。颜色对应于四旋翼飞行器的速度。

特定类别的轨道，直到达到指定的性能阈值。赛道复杂性的适应可以解释为自动课程，从而产生针对代理能力量身定制的学习过程。

四. 实验

我们设计实验来回答以下研究问题：(i) 与确定性赛道布局的优化算法相比，我们基于学习的方法实现的单圈时间如何？(ii) 我们的方法能够如何很好地处理轨道布局的变化？(iii) 是否有可能训练出能够在完全未知的赛道上成功比赛的策略？(iv) 我们的策略生成的轨迹可以用物理四旋翼飞行器执行吗？

A. Experimental Setup

我们使用 *Flightmare* 模拟器 [27] 并实现矢量化 *OpenAI* 健身房风格的赛车环境，该环境允许并行模拟数百个四旋翼飞行器和赛道。这种并行化实现使我们能够在训练期间每秒收集多达 25000 次环境交互。我们的 PPO 实现基于 [28]。

为了对我们的方法的性能进行基准测试，我们使用了三个不同的赛道，包括来自 2019 年 *AlphaPilot* 挑战赛的赛道 [8]、来自 2019 年 *NeurIPS AirSim* 无人机游戏挑战赛的赛道 [7] 以及为真实世界实验设计的赛道 (*Split-S*)。这些赛道 (图 5) 为自动竞赛无人机带来了一系列不同的挑战：*AlphaPilot* 赛道具有长直线段和发夹弯道，*AirSim* 赛道需要执行快速且非常大的海拔变化，而 *Split-S* 赛道引入了两个垂直堆叠的门。

为了测试我们方法的鲁棒性，我们使用 *AlphaPilot* 轨道上所有门的位置和偏航角的随机位移。此外，我们使用第 III-C.3 节中介绍的轨迹生成器在完全随机的轨迹上训练和评估我们的方法，使我们能够研究我们方法的可扩展性和通用性。最后，我们通过消融研究验证我们的设计选择，并在物理平台上执行生成的轨迹。所有实验均使用四旋翼配置进行，其对角惯量矩阵为 $\mathbf{J} = [0.003, 0.003, 0.005] \text{ kg m}^2$ ，扭矩常数为 $\kappa = 0.01$ ，臂长为 0.17 m。对于

AlphaPilot 对于 *AirSim* 轨道，我们使用 6.4 的推力重量比，而对于 *Split-S* 轨道，使用 3.3 的推力重量比（用于实际部署）。除非另有说明，我们在观察中使用两个未来的门来执行所有实验。我们在第 IV-E.2 节中提出了关于门数量的消融研究。

B. Baseline Comparison on Deterministic Tracks

我们首先针对两种轨迹规划算法在三个确定性轨迹（第 IV-A 节）上对我们的策略的性能进行基准测试：多项式最小捕捉轨迹生成 [29] 和基于互补进度约束 (CPC) [1] 的基于优化的时间最优轨迹生成。多项式轨迹生成，以及最小捕捉轨迹生成的具体实例，由于其隐式平滑特性而被广泛用于四旋翼飞行。尽管总体来说这是一个非常有用的功能，但平滑度属性阻止了在每个时间点充分利用执行器的潜力，从而使该方法对于赛车来说不是最佳的。相比之下，基于优化的方法 (CPC) 为给定的赛道和车辆模型提供了可能实现的时间的渐近下限。我们不与基于采样的方法进行比较，因为它们简化了四旋翼动力学的假设。据我们所知，不存在考虑四旋翼飞行器完整状态空间的基于采样的方法。

结果总结在表 I 中。我们的方法的性能与 CPC 所示的理论极限相差不超过 5.2%。使用多项式轨迹生成计算的基线要慢得多。图 5 显示了三个评估的赛道以及我们的方法生成的相应轨迹。

表 I: 单圈时间（以秒为单位）。

Methods	AlphaPilot (s)	Split-S (s)	AirSim (s)
Polynomial [29]	12.23	15.13	23.82
Optimization [1]	8.06	6.18	11.40
Ours	8.14	6.50	11.82

C. Handling Race Track Changes

While the results on the deterministic track layouts allow to compare the racing performance to the theoretical limit, such lap times are impossible to obtain in a more realistic setting where the track layout is not exactly known in advance.

We design an experiment to study the robustness of our approach to partially unknown track layouts by specifying random gate displacements on the *AlphaPilot* track from the previous experiment. Specifically, we use bounded displacements $[\Delta x, \Delta y, \Delta z, \Delta \alpha]$ on the position of each gate's center as well as the gate's orientation in the horizontal plane. Fig. 6 illustrates the significant range of displacements that are applied to the original track layout.

We train a policy from scratch by sampling different gate layouts from a uniform distribution of the individual displacement bounds. In total, we create 100 parallel racing environments with different track configurations for each training iteration. The final model is selected based on the fastest lap time on the nominal version of the *AlphaPilot* track. We define two metrics to evaluate the performance of the trained policy: the average lap time and the crash ratio. We compute both metrics on three difficulty levels (defined by the randomness of the track), each comprising a set of 1000 unseen randomly-generated track configurations.

The results are shown in Table II. The trained policy achieves a success rate of 97.5% on the most difficult test set which features maximum gate displacements. The lap time performance on the nominal track decreased by 2.21% compared to the policy trained without displacements (Table I). To illustrate the complexity of the task, Fig. 6 shows

TABLE II: Evaluation results of handling track uncertainties. The gate randomization level corresponds to the percentage of the maximum gate displacements.

Gate Randomization Level	Avg. Time (s)	#Crashes
0.00	8.32	0
0.50	8.43	0
0.75	8.55	2
1.00	8.73	25

100 successful trajectories on randomly sampled tracks using maximum gate displacements along with the nominal trajectory.

D. Towards Learning a Universal Racing Policy

We study the scalability and generalizability of our approach by designing an experiment to train a policy that allows us to generate time-optimal trajectories for completely unseen tracks.

To this end, we train three different policies on completely random track layouts in each rollout. The three policies differ in the training strategy and the reward used: we ablate the usage of the automatic track adaptation and the safety reward. During training, we continuously evaluate the performance of

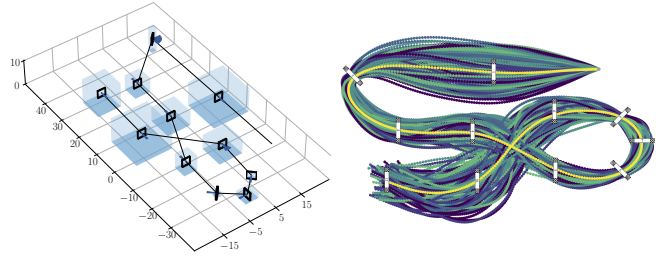


Fig. 6: *Left*: Track changes for the *AlphaPilot* track. The gate center can be moved to arbitrary positions within the respective cube while possible orientations are illustrated by the blue cones. *Right*: Trajectories on different configurations of the *AlphaPilot* track. The color scheme is chosen to highlight different trajectories and does not reflect any metric related to the trajectory. The nominal trajectory is displayed in yellow.

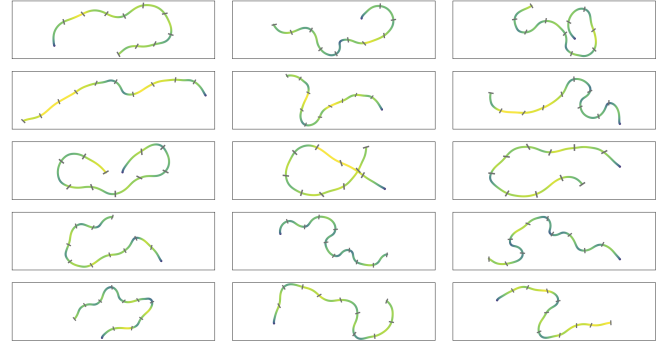


Fig. 7: Policy rollouts on randomly generated tracks (visualization only in xy -plane). The trajectories range from 110 m to 150 m length with elevation changes of up to 17 m.

the current policy on a validation set of 100 unseen randomly sampled tracks. For each configuration we choose the policy with the lowest crash ratio on the validation set as our final model. The final evaluation is performed on a test set of 1000 randomly sampled tracks of full complexity and we assess the performance based on the number of gate crashes.

TABLE III: Performance on randomly generated race tracks. We report the number of crashes on 1000 test tracks.

Safety Reward	Track Adaptation	#Crashes
✗	✗	71
✗	✓	47
✓	✓	26

The results of the experiment are displayed in Table III. When training the policy without both, the safety reward and the automatic track adaptation, the policy performs worst in terms of number of crashes. Adding automatic track adaptation considerably improves performance by reducing the task complexity at the beginning of the training process and taking into account the current learning progress when sampling the random tracks. By adding the safety reward, we manage to further reduce the crash ratio, illustrating the

C. Handling Race Track Changes

虽然确定性赛道布局的结果允许将赛车性能与理论极限进行比较，但在无法提前准确了解赛道布局的更现实的环境中，不可能获得这样的单圈时间。

我们设计了一个实验，通过指定先前实验中 *AlphaPilot* 轨道上的随机门位移来研究我们的方法对部分未知轨道布局的鲁棒性。具体来说，我们在每个门的中心位置以及门在水平面上的方向使用有界位移 $[\Delta x, \Delta y, \Delta z, \Delta \alpha]$ 。图 6 说明了应用于原始轨道布局的位移的显着范围。

我们通过从各个位移边界的均匀分布中采样不同的门布局来从头开始训练策略。总共，我们为每次训练迭代创建了 100 个具有不同赛道配置的并行赛车环境。最终模型是根据 *AlphaPilot* 赛道标称版本上的最快单圈时间选择的。我们定义了两个指标来评估训练策略的性能：平均单圈时间和碰撞率。我们在三个难度级别（由赛道的随机性定义）上计算这两个指标，每个难度级别包含一组 1000 个未见过的随机生成的赛道配置。

结果如表II所示。经过训练的策略在具有最大门位移的最困难的测试集上实现了 97.5% 的成功率。与没有位移训练的策略相比，标称赛道上的单圈时间性能下降了 2.2 1%（表 I）。为了说明任务的复杂性，图 6 显示

表二：处理轨迹不确定性的评估结果。门随机化水平对应于最大门位移的百分比。

Gate Randomization Level	Avg. Time (s)	#Crashes
0.00	8.32	0
0.50	8.43	0
0.75	8.55	2
1.00	8.73	25

使用最大闸门位移和标称轨迹在随机采样轨道上获得 10 0 个成功轨迹。

D. Towards Learning a Universal Racing Policy

我们通过设计实验来训练策略来研究我们方法的可扩展性和通用性，该策略允许我们为完全看不见的轨迹生成时间最优轨迹。

为此，我们在每次部署中在完全随机的轨道布局上训练三种不同的策略。这三种策略在训练策略和所使用的奖励方面有所不同：我们取消了自动赛道适应和安全奖励的使用。在培训期间，我们不断评估员工的表现

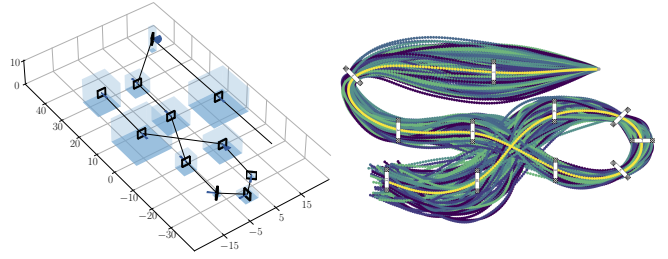


图 6: *Left*: 轨道对 *AlphaPilot* 轨道的更改。门中心可以移动到相应立方体内的任意位置，而可能的方向由蓝色锥体表示。 *Right*: *AlphaPilot* 轨道不同配置上的轨迹。选择颜色方案是为了突出显示不同的轨迹，并且不反映与轨迹相关的任何度量。标称轨迹显示为黄色。

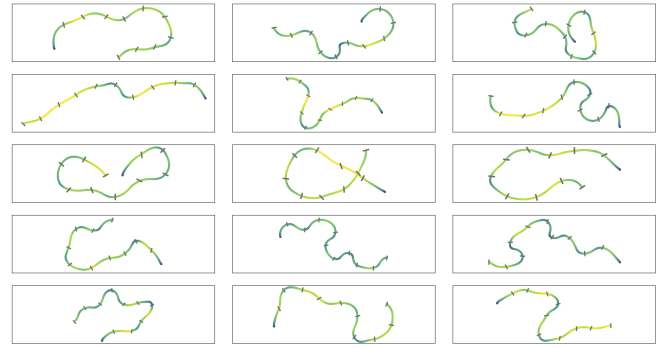


图 7：在随机生成的轨道上推出策略（仅在 xy - 平面中可视化）。轨迹长度范围为 110 m 至 150 m，海拔变化可达 17 m。

当前策略是针对 100 个未见过的随机采样曲目的验证集。对于每个配置，我们选择验证集上崩溃率最低的策略作为最终模型。最终评估是在 1000 个完全复杂的随机采样轨道的测试集上进行的，我们根据门碰撞次数评估性能。

表 III：随机生成的赛道上的表现。我们报告了 1000 条测试跑道上的碰撞次数。

Safety Reward	Track Adaptation	#Crashes
✗	✗	71
✗	✓	47
✓	✓	26

实验结果显示在表III中。当在没有安全奖励和自动赛道适应的情况下训练策略时，该策略在碰撞次数方面表现最差。添加自动轨迹自适应可以降低训练过程开始时的任务复杂性，并在对随机轨迹进行采样时考虑当前的学习进度，从而显着提高性能。通过添加安全奖励，我们成功地进一步降低了事故率，说明了

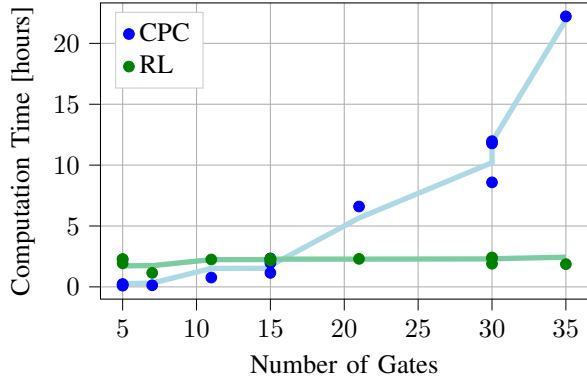


Fig. 8: Computation time comparison between our learning-based method and the optimization-based method (CPC).

positive effect of maximizing the safety margins when training on random tracks. Our policy achieves 97.4% success rate on 1000 randomly generated tracks, exhibiting strong generalizability and scalability.

To illustrate the large variety of the randomly sampled tracks, Fig. 7 shows a subset of the evaluation trajectories.

E. Computation Time Comparison and Ablation Studies

1) *Computation Time Comparison*: We compare the training time of our reinforcement learning approach to the computation time of the trajectory optimization method in a controlled experiment. Specifically, we compare the computation times for the three trajectories presented in Section IV-B, which consists of 7, 11, and 21 gates (for SplitS, AlphaPilot, and AirSim respectively). Also, we evaluate the computation time on randomly generated race tracks consisting of 5, 15, 30 and 35 gates. For each number of gates we run 3 different experiments and compute the mean computation time. We only conduct a single experiment for 35 gates due to the long computation time of the optimization-based approach. Consequently, we report the computation time for a total of 13 challenging race tracks.

The results of this experiment are displayed in Figure 8. The results show that the computation time of the optimization-based method increases drastically with the number of gates as the number of decision variables in the optimization problem scales accordingly. The training time of our learning-based method instead does not depend on the number of gates, making the approach particularly appealing for large-scale environments.

2) *Ablation Studies*: We perform an ablation study to validate design choices of the proposed approach. Specifically, we focus on the effect of different observation models as well as the safety reward.

Number of Observed Gates: Since the number of future gates included in the observation corresponds to neither a fixed distance nor time, we consider it necessary to ablate the effect of the number of gates on the policy’s performance.

We train policies on the *AlphaPilot* track using 1, 2 and 3 future gates in the observation. Both versions of the track, the deterministic layout as well as the layout subject to bounded displacements presented in Section IV-C are used. The final model for evaluation is selected based on the fastest lap time

achieved on the nominal track. The results of this experiment are displayed in Table IV.

TABLE IV: Results of the gate observation ablation experiment. We report the lap time on the nominal track as well as average lap time and crash ratio on 1000 test tracks.

#Gates	Lap Time	Avg. Lap Time (s) / Crash Ratio (%)
1	8.20	9.92 / 23.0
2	8.14	8.32 / 2.5
3	8.16	8.36 / 2.3

We show that the final lap time on the deterministic track is not affected significantly by the number of observed gates. This can be explained by the static nature of the task for which it is possible to overfit to the track layout even without information beyond the next gate to be passed. However, when introducing gate displacements, using only information about a single future gate is insufficient to achieve fast lap times and minimize the crash ratio.

Safety Reward: We introduce the safety reward in order to alter the performance objective such that the distance to the gate center at the time of the gate crossing is minimized during training. This becomes relevant in training settings with significant track randomization as well as real-world experiments that require safety margins due to uncertain effects of the physical system. We design our experiments to evaluate the impact of two safety reward configurations on the number of crashes as well as the safety margins of successful gate crossings.

TABLE V: Results of the safety reward ablation experiment. We report the lap time on the nominal track layout, the crash ratio on 1000 test tracks and statistics about the safety margins of successful gate crossings.

d_{max}	Lap Time (s)	Crash Ratio (%)	Safety Margins (m)
0.0	8.32	2.5	0.67 ± 0.28
2.5	8.42	0.8	0.95 ± 0.34
5.0	8.48	0.6	0.94 ± 0.35

The results of this experiment are displayed in Table V. We show that the safety reward increases the average safety margin for all configurations. Furthermore, the results indicate that the safety reward consistently reduces the number of crashes on the test set. However, both configurations of the safety reward result in reduced performance on the nominal track.

F. Real-world Flight

Finally, we execute the trajectory generated by our policy on a physical quadrotor to verify the transferability of our policy. We use a quadrotor that is developed from off-the-shelf components made for drone racing, including a carbon-fiber frame, brushless DC motors, 5-inch propellers, and a BetaFlight controller. The quadrotor has a thrust-to-weight

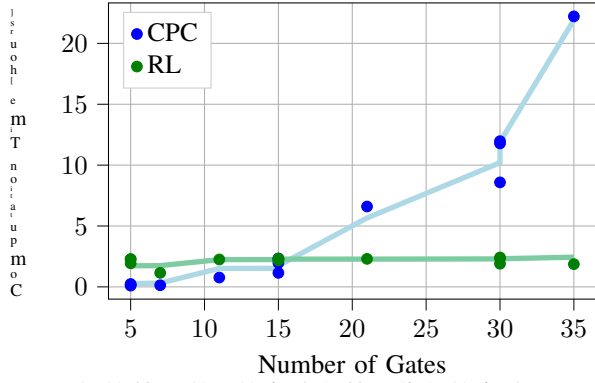


图 8: 我们的基于学习的方法和基于优化的方法 (CPC) 之间的计算时间比较。

在随机轨道上训练时最大化安全裕度的积极影响。我们的策略在 1000 个随机生成的轨迹上实现了 97.4% 的成功率，表现出很强的通用性和可扩展性。

为了说明随机采样轨迹的多样性，图 7 显示了评估轨迹的子集。

E. Computation Time Comparison and Ablation Studies

1) *Computation Time Comparison*: 我们在受控实验中将强化学习方法的训练时间与轨迹优化方法的计算时间进行了比较。具体来说，我们比较了第 IV-B 节中提出的三个轨迹的计算时间，这三个轨迹由 7、11 和 21 个门组成（分别针对 SplitS、AlphaPilot 和 AirSim）。此外，我们还评估了随机生成的由 5、15、30 和 35 个门组成的赛道上的计算时间。对于每个门数，我们运行 3 个不同的实验并计算平均计算时间。由于基于优化的方法的计算时间较长，我们只对 35 个门进行了一次实验。因此，我们报告了总共 13 条具有挑战性的赛道的计算时间。

该实验的结果如图 8 所示。结果表明，随着优化问题中决策变量数量的增加，基于优化的方法的计算时间随着门的数量而急剧增加。相反，我们基于学习的方法的训练时间并不取决于门的数量，这使得该方法对于大规模环境特别有吸引力。

2) *Ablation Studies*: 我们进行了一项消融研究来验证所提出方法的设计选择。具体来说，我们关注不同观察模型的效果以及安全奖励。

Number of Observed Gates: 由于观察中包含的未来门的数量既不对应于固定距离，也不对应于固定时间，因此我们认为有必要消除门数量对策略性能的影响。

我们在观察中使用 1、2 和 3 个未来门在 AlphaPilot 轨道上训练策略。两种版本的轨道、确定性布局以及受第 IV-C 节中介绍的有限位移影响的布局均被使用。根据最快单圈时间选择最终的评估模型

在名义轨道上实现。该实验的结果显示在表 IV 中。

表 IV: 门观察消融实验的结果。我们报告标称赛道上的单圈时间以及 1000 条测试赛道上的平均单圈时间和碰撞率。

#Gates	Lap Time	Avg. Lap Time (s) / Crash Ratio (%)
1	8.20	9.92 / 23.0
2	8.14	8.32 / 2.5
3	8.16	8.36 / 2.3

我们表明，确定性赛道上的最终单圈时间不会受到观察到的门的数量的显着影响。这可以通过任务的静态性质来解释，即使没有要通过的下一个门之外的信息，也可能会过度拟合轨道布局。然而，在引入浇口位移时，仅使用有关单个未来浇口的信息不足以实现快速单圈时间并最大限度地降低碰撞率。

Safety Reward: 我们引入安全奖励是为了改变性能目标，以便在训练期间最小化到门中心的距离。这在具有显着轨迹随机化的训练环境以及由于物理系统的不确定影响而需要安全裕度的现实世界实验中变得相关。我们设计实验来评估两种安全奖励配置对碰撞次数以及成功通过大门的安全裕度的影响。

表 V: 安全奖励消融实验的结果。我们报告标称赛道布局的单圈时间、1000 条测试赛道的碰撞率以及成功穿越大门的安全裕度的统计数据。

d_{max}	Lap Time (s)	Crash Ratio (%)	Safety Margins (m)
0.0	8.32	2.5	0.67 $\pm$ 0.28
2.5	8.42	0.8	0.95 $\pm$ 0.34
5.0	8.48	0.6	0.94 $\pm$ 0.35

该实验的结果如表 V 所示。我们表明，安全奖励增加了所有配置的平均安全裕度。此外，结果表明，安全奖励持续减少了测试集上的崩溃次数。然而，这两种安全奖励配置都会导致名义赛道上的表现下降。

F. Real-world Flight

最后，我们在物理四旋翼飞行器上执行我们的策略生成的轨迹，以验证我们的策略的可转移性。我们使用的四旋翼飞行器是由专为无人机竞赛制造的现成组件开发而成，包括碳纤维框架、无刷直流电机、5 英寸螺旋桨和 BetaFlight 控制器。四旋翼飞行器具有推重比



Fig. 9: A composite image showing the real-world flight.

ratio of around 4. A reference trajectory is generated by performing a deterministic policy rollout for four continuous laps on the *Split-S* track. This reference trajectory is then tracked by a model predictive controller. Fig. 9 shows an image of the real-world flight. Our trajectory enables the vehicle to fly at high speeds of up to 60 km h^{-1} , pushing the platform to its physical limits. However, executing the trajectory results in large tracking errors. Future research will focus on improving the time-optimal trajectory tracking performance.

V. CONCLUSION

In this work, we presented a learning-based method for training a neural network policy that can generate near-time-optimal trajectories through multiple gates for quadrotors. We demonstrated the strengths of our approach, including near-time-optimal performance, the capability of handling large track changes, and the scalability and generalizability to tackle large-scale random track layouts while retaining computational efficiency. We validated the generated trajectory with a physical quadrotor and achieved aggressive flight at speeds of up to 60 km h^{-1} . These findings suggest that deep RL has the potential for generating adaptive time-optimal trajectories for quadrotors and merits further investigation.

ACKNOWLEDGEMENT

The authors thank Philipp Foehn and Angel Romero for helping with the real-world experiments and providing the trajectory optimization baseline. Also, the authors thank Prof. Jürgen Rossmann and Alexander Atanasyan of the Institute for Man-Machine Interaction, RWTH Aachen University for their helpful advice.

REFERENCES

- [1] P. Foehn, A. Romero, and D. Scaramuzza, "Time-optimal planning for quadrotor waypoint flight," *Science Robotics*, 2021.
- [2] G. Loianno, C. Brunner, G. McGrath, and V. Kumar, "Estimation, control, and planning for aggressive flight with a small quadrotor with a single camera and imu," *IEEE Rob. and Autom. Letters*, 2017.
- [3] E. Kaufmann, A. Loquercio, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Deep drone acrobatics," *RSS: Robotics, Science, and Systems*, 2020.
- [4] H. Moon, J. Martinez-Carranza, T. Cieslewski, M. Faessler, D. Falanga, A. Simovic, D. Scaramuzza, S. Li, M. Ozo, C. De Wagter *et al.*, "Challenges and implemented technologies used in autonomous drone racing," *Intelligent Service Robotics*, 2019.
- [5] J. A. Cocomo-Ortega and J. Martínez-Carranza, "Towards high-speed localisation for autonomous drone racing," in *Mexican International Conference on Artificial Intelligence*. Springer, 2019.
- [6] E. Kaufmann, M. Gehrig, P. Foehn, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, "Beauty and the beast: Optimal methods meet learning for drone racing," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 690–696.

- [7] R. Madaan, N. Gyde, S. Vemprala, M. Brown, K. Nagami, T. Taubner, E. Cristofalo, D. Scaramuzza, M. Schwager, and A. Kapoor, "Airsim drone racing lab," in *NeurIPS 2019 Competition and Demonstration Track*. PMLR, 2020.
- [8] W. Guerra, E. Tal, V. Murali, G. Ryou, and S. Karaman, "FlightGoggles: Photorealistic sensor simulation for perception-driven robotics using photogrammetry and virtual reality," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019.
- [9] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar, and D. Scaramuzza, "Alphapilot: Autonomous drone racing," *RSS: Robotics, Science, and Systems*, 2020.
- [10] G. Ryou, E. Tal, and S. Karaman, "Multi-fidelity black-box optimization for time-optimal quadrotor maneuvers," *RSS: Robotics, Science, and Systems*, 2020.
- [11] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science robotics*, vol. 5, no. 47, 2020.
- [12] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, "Control of a quadrotor with reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 2, no. 4, pp. 2096–2103, 2017.
- [13] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017.
- [14] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The international journal of robotics research*, 2011.
- [15] D. J. Webb and J. Van Den Berg, "Kinodynamic rrt*: Asymptotically optimal motion planning for robots with linear dynamics," in *IEEE International Conference on Robotics and Automation*. IEEE, 2013.
- [16] R. E. Allen and M. Pavone, "A real-time framework for kinodynamic planning in dynamic environments with application to quadrotor obstacle avoidance," *Robotics and Autonomous Systems*, vol. 115, pp. 174–193, 2019.
- [17] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and efficient quadrotor trajectory generation for fast autonomous flight," *IEEE Robotics and Automation Letters*, 2019.
- [18] S. Spedicato and G. Notarstefano, "Minimum-time trajectory generation for quadrotors in constrained environments," *IEEE Transactions on Control Systems Technology*, vol. 26, no. 4, pp. 1335–1344, 2017.
- [19] F. Fuchs, Y. Song, E. Kaufmann, D. Scaramuzza, and P. Duerr, "Superhuman performance in gran turismo sport using deep reinforcement learning," *IEEE Robotics and Automation Letters*, 2021.
- [20] M. Jaritz, R. De Charette, M. Toromanoff, E. Perot, and F. Nashashibi, "End-to-end race driving with deep reinforcement learning," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 2070–2075.
- [21] S. Aradi, "Survey of deep reinforcement learning for motion planning of autonomous vehicles," *IEEE Transactions on Intelligent Transportation Systems*, 2020.
- [22] N. Roy and S. Thrun, "Motion planning through policy search," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, vol. 3. IEEE, 2002, pp. 2419–2424.
- [23] A. Rodriguez-Ramos, C. Sampedro, H. Bavl, P. De La Puente, and P. Campoy, "A deep reinforcement learning strategy for uav autonomous landing on a moving platform," *Journal of Intelligent & Robotic Systems*, 2019.
- [24] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. The MIT Press, 2018.
- [25] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale rc cars," *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, 2015.
- [26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [27] Y. Song, S. Naji, E. Kaufmann, A. Loquercio, and D. Scaramuzza, "Flightmare: A flexible quadrotor simulator," in *Conference on Robot Learning (CoRL)*, 2020.
- [28] A. Raffin, A. Hill, M. Ernestus, A. Gleave, A. Kanervisto, and N. Dormann, "Stable baselines3," <https://github.com/DLR-RM/stable-baselines3>, 2019.
- [29] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 2520–2525.

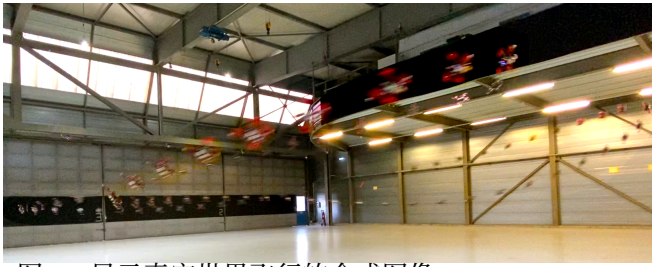


图 9: 显示真实世界飞行的合成图像。

比率约为 4。通过在 *Split-S* 轨道上连续四圈执行确定性策略推出来生成参考轨迹。然后由模型预测控制器跟踪该参考轨迹。图 9 显示了真实世界飞行的图像。我们的轨迹使车辆能够以高达 60 km h^{-1} 的高速飞行，将平台推向其物理极限。然而，执行轨迹会导致较大的跟踪误差。未来的研究将集中于提高时间最优轨迹跟踪性能。

五、结论

在这项工作中，我们提出了一种基于学习的方法来训练神经网络策略，该策略可以通过四旋翼飞行器的多个门生成接近时间最优的轨迹。我们展示了我们方法的优势，包括近乎最佳的性能、处理大型轨道变化的能力，以及在保持计算效率的同时处理大规模随机轨道布局的可扩展性和通用性。我们使用物理四旋翼飞行器验证了生成的轨迹，并实现了速度高达 60 km h^{-1} 的激进飞行。这些发现表明深度强化学习有潜力为四旋翼飞行器生成自适应时间最优轨迹，值得进一步研究。

致谢

作者感谢 Philipp Foehn 和 Angel Romero 帮助进行现实世界的实验并提供轨迹优化基线。此外，作者还感谢亚琛工业大学人机交互研究所的 Jurgen Rossmann 教授和 Alexander Atanasyan 教授提供的有益建议。

参考

[1] P. Foehn, A. Romero 和 D. Scaramuzza, “四旋翼飞行器航路点飞行的时间最优规划”, *Science Robotics*, 2021 年。[2] G. Loianno, C. Brunner, G. McGrath 和 V. Kumar, “使用单个摄像头和 IMU 的小型四旋翼飞行器进行激进飞行的估计、控制和规划”, *IEEE Rob. and Autom. Letters*, 2017 年。[3] E. Kaufmann, A. Loquercio, R. Ranftl, M. Müller, V. Koltun 和 D. Scaramuzza, “深度无人机杂技”, *RSS: Robotics, Science, and Systems*, 2020。[4] H. Moon, J. Martinez-Carranza, T. Cieslewski, M. Faessler, D. Falanga, A. Simovic, D. Scaramuzza, S. Li, M. Ozo, C. De Wagter *et al.*, “自主无人机竞赛中使用的挑战和实施技术”, *Intelligent Service Robotics*, 2019 年。[5] J. A. Cocoma-Ortega 和 J. Martínez-Carranza, “实现自主无人机竞赛的高速本地化”, *Mexican International Conference on Artificial Intelligence*. Springer, 2019。[6] E. Kaufmann, M. Gehrig, P. Foehn, R. Ranftl, A. Dosovitskiy, V. Koltun 和 D. Scaramuzza, “美女与野兽：无人机竞赛的最佳方法中学习”, 载于 *2019 International Conference on Robotics and Automation (ICRA)*。IEEE, 2019 年, 第 690–696 页。

[7] R. Madaan, N. Gyde, S. Vemprala, M. Brown, K. Nagami, T. T. T. Taubner, E. Cristofalo, D. Scaramuzza, M. Schwager 和 A. Kapoor, “Airsim 无人机赛车实验室”, *NeurIPS 2019 Competition and Demonstration Track*. PMLR, 2020。[8] W. Guerra, E. Tal, V. Murali, G. Ryou 和 S. Karaman, “FlightGoggles: 使用摄影测量和虚拟现实进行感知驱动机器人的真实感传感器模拟”, *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019。[9] P. Foehn, D. Brescianini, E. Kaufmann, T. Cieslewski, M. Gehrig, M. Muglikar 和 D. Scaramuzza, “Alphapilot: 自主无人机竞赛”, *RSS: Robotics, Science, and Systems*, 2020。[10] G. Ryou, E. Tal 和 S. Karaman, “多保真度黑盒优化”时间最佳的四旋翼机动, ” *RSS: Robotics, Science, and Systems*, 2020。[11] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun 和 M. Hutter, “在具有挑战性的地形上学习四足运动”, *Science robotics*, 卷。5、没有。47, 2020。[12] J. Hwangbo, I. Sa, R. Siegwart 和 M. Hutter, “通过强化学习控制四旋翼飞行器”, *IEEE Robotics and Automation Letters*, 卷。2、没有。4, 第 2096–2103 页, 2017 年。[13] S. Gu, E. Holly, T. Lillicrap 和 S. Levine, “具有异步策略更新的机器人操作的深度强化学习”, 见 *2017 IEEE international conference on robotics and automation (ICRA)*。IEEE, 2017。[14] S. Karaman 和 E. Frazzoli, “基于采样的最优运动规划算法”, *The international journal of robotics research*, 2011。[15] D. J. Webb 和 J. Van Den Berg, “Kinodynamic rrt*: 线性动力学机器人的渐进最优运动规划”, *IEEE International Conference on Robotics and Automation*. IEEE, 2013。[16] R. E. Allen 和 M. Pavone, “动态环境中运动动力学规划的实时框架及其应用于四旋翼飞行器避障”, *Robotics and Autonomous Systems*, 卷。115, 第 174–193 页, 2019 年。[17] B. Zhou, F. Gau, L. Wang, C. Liu 和 S. Shen, “用于快速自主飞行的稳健且高效的四旋翼轨迹生成”, *IEEE Robotics and Automation Letters*, 2019。[18] S. Spedicato 和 G. Notarstefano, “最短时间轨迹生成”对于受限环境中的四旋翼飞行器, ” *IEEE Transactions on Control Systems Technology*, 卷。26、没有。4, 第 1335–1344 页, 2017 年。[19] F. Fuchs, Y. Song, E. Kaufmann, D. Scaramuzza 和 P. Duerr, “利用深度强化学习在大型赛车运动中实现超人表现”, *IEEE Robotics and Automation Letters*, 2021 年。[20] M. Jaritz, R. De Charette, M. Toromanoff, E. Perot 和 F. Nashashibi, “利用深度强化学习的端到端赛车驾驶”, *2018 IEEE International Conference on Robotics and Automation (ICRA)*。IEEE, 2018 年, 第 2070–2075 页。[21] S. Aradi, “自动驾驶车辆运动规划深度强化学习调查”, *IEEE Transactions on Intelligent Transportation Systems*, 2020。[22] N. Roy 和 S. Thrun, “通过策略搜索进行运动规划”, *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 卷。3. IEEE, 2002 年, 第 2419–2424 页。[23] A. Rodriguez-Ramos, C. Sampedro, H. Bavle, P. De La Puente 和 P. Campoy, “无人机在移动平台上自主着陆的深度强化学习策略”, *Journal of Intelligent & Robotic Systems*, 2019。[24] R. S. Sutton 和 A. G. Barto, *Reinforcement Learning: An Introduction*, 第二版。麻省理工学院出版社, 2018 年。[25] A. Liniger, A. Domahidi 和 M. Morari, “基于优化的 1:43 比例遥控汽车自主赛车”, *Optimal Control Applications and Methods*, 卷。36、没有。5, 第 628–647 页, 2015 年。[26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford 和 O. Klimov, “近端策略优化算法” *arXiv preprint arXiv:1707.06347*, 2017 年。[27] Y. Song, S. Naji, E. Kaufmann, A. Loquercio 和 D. Scaramuzza, “Flightmare: 灵活的四旋翼模拟器”, *Conference on Robot Learning (CoRL)*, 2020 年。[28] A. Raffin, A. Hill, M. Ernestus, A. Gleave, A. Kanervisto 和 N. Dormann, “稳定基线3”, <https://github.com/DLR-RM/stable-baselines3>, 2019 年。[29] D. Mellinger 和 V. Kumar, “四旋翼飞行器的最小捕捉轨迹生成和控制”, 参见 *2011 IEEE international conference on robotics and automation*。IEEE, 2011 年, 第 2520–2525 页。